



CSE 711 - Compilers

Lecture 1

Course Instructor: Ms. Samia Muntaha
Department of Computer Science and Engineering
University of Chittagong

[Researchgate Profile](#)

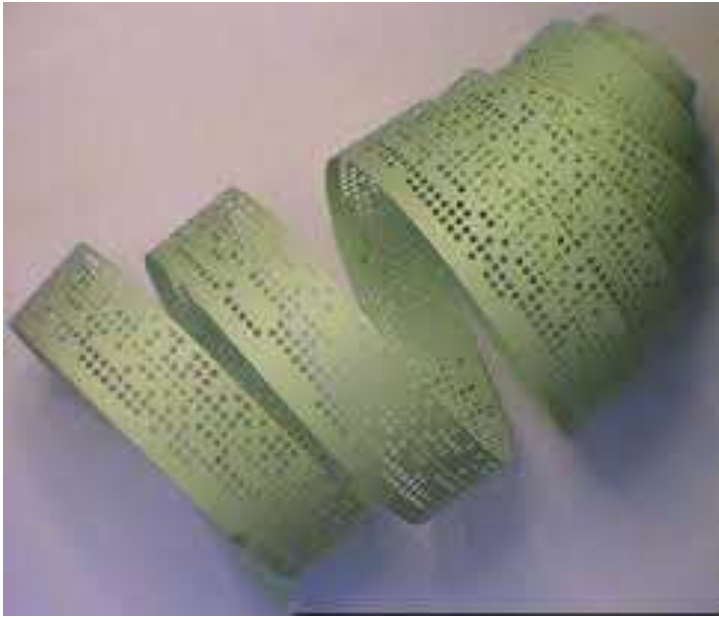
Course Information

- ▶ **Prerequisites:** CSE 611 (Theory of Computation)
- ▶ Google Classroom Link:
<https://classroom.google.com/c/ODQwMDg2ODU1OTYy?cjc=dyaaf37p>
- ▶ Textbook:
 1. Compilers: Principles, Techniques, and Tools by Aho, Lam, Sethi, Ullman
 2. Modern Compiler Implementation in C (The Tiger Book) by Andrew W. Appel

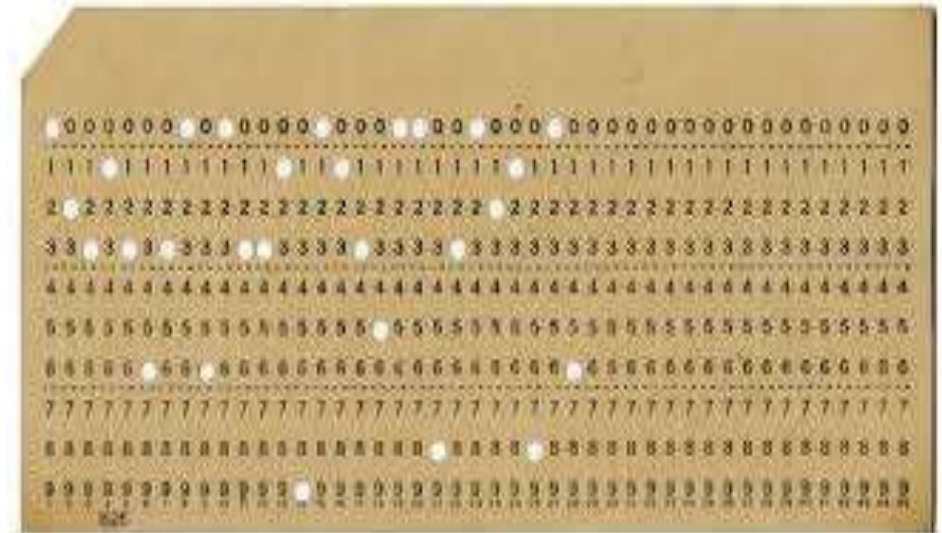
Objectives

- ▶ Language translators
- ▶ Compiler vs Interpreter
- ▶ Language processing system
- ▶ Overview of different phases of a compiler

Language Translator

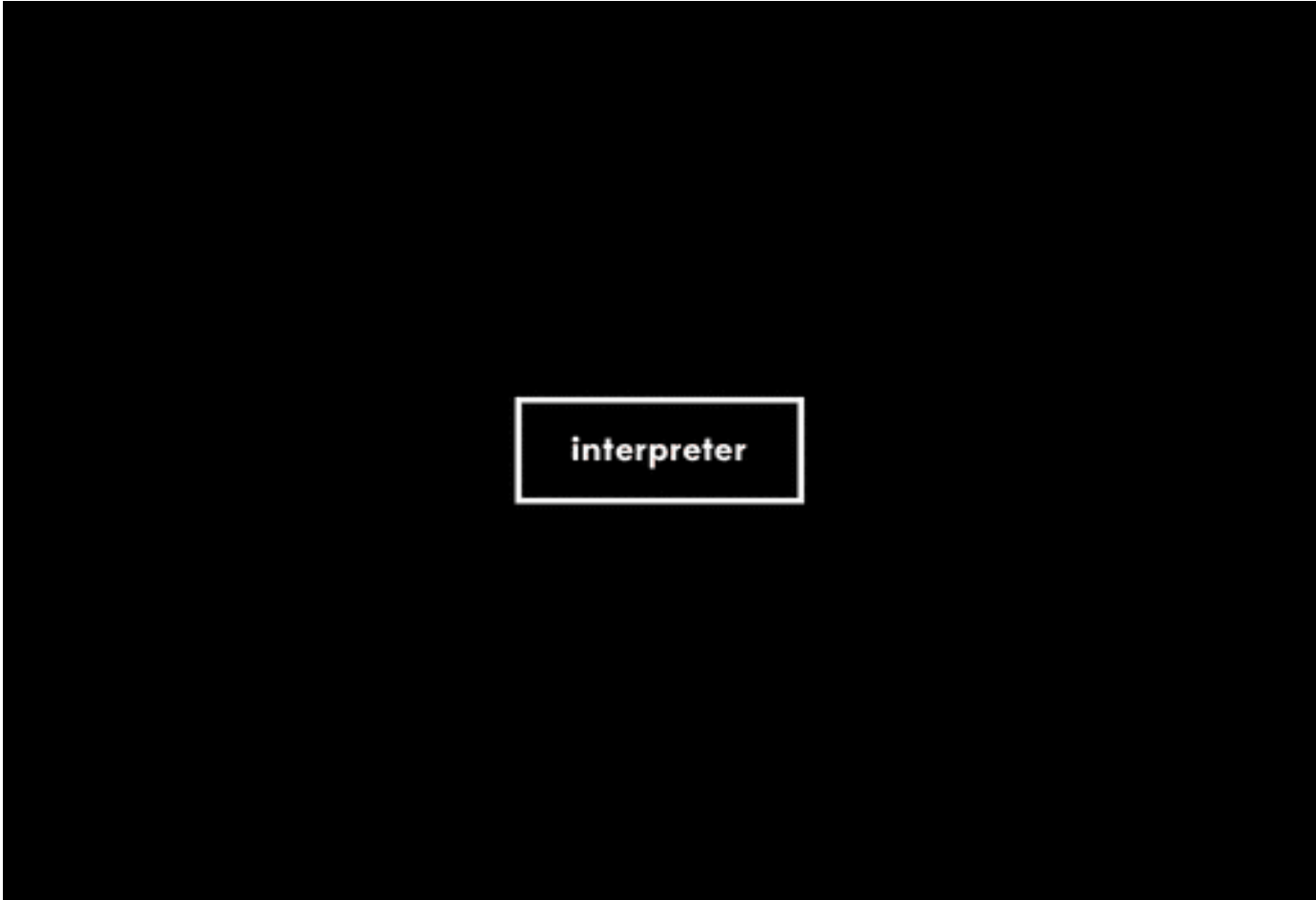


Paper Tape



Punched Card

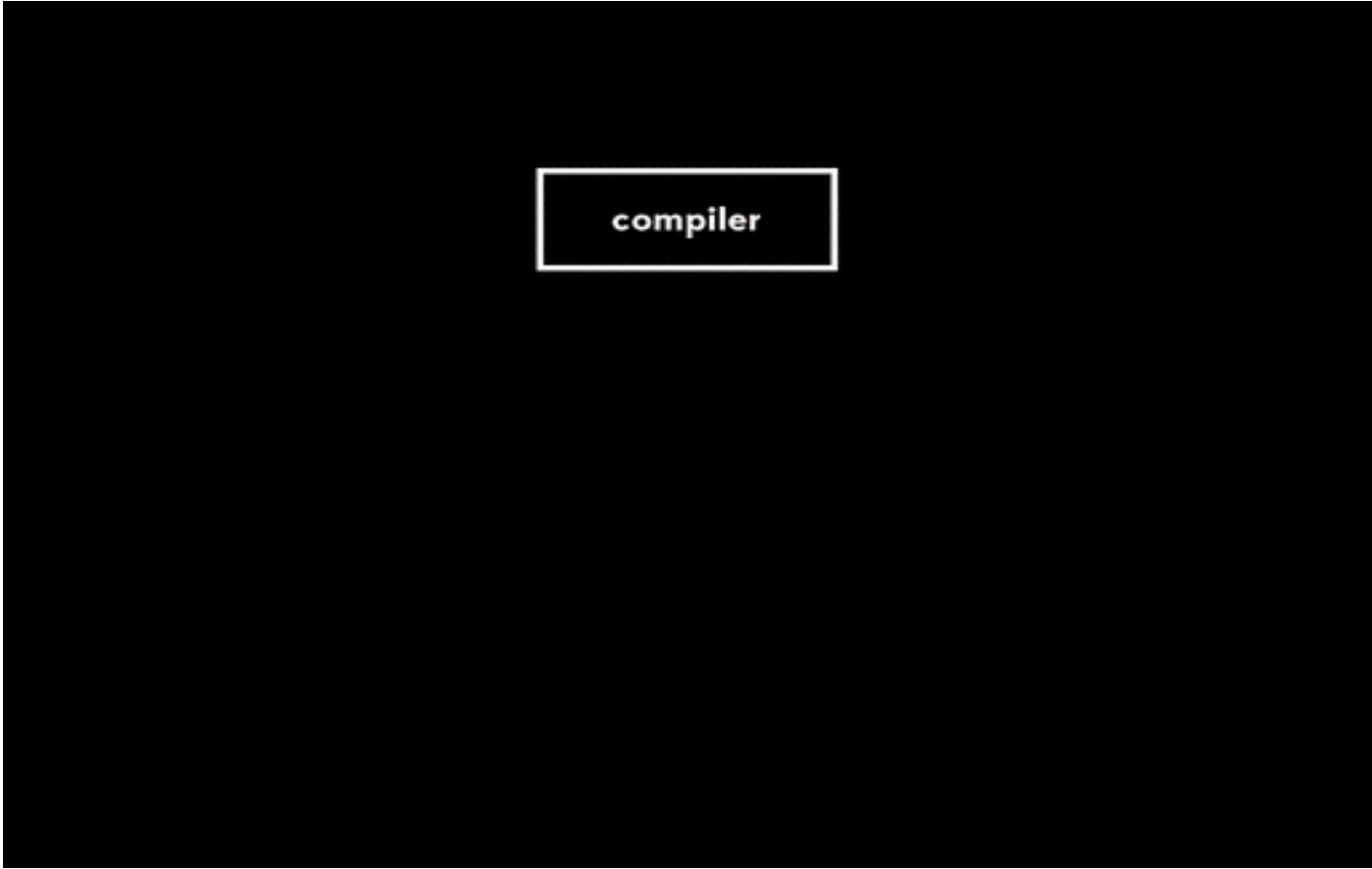
Compiler Vs Interpreter



interpreter



Compiler Vs Interpreter



compiler



High Level Language

```
#include <stdio.h> //header file for printf()

int main() { //main function
    int a=2, b=3, c=5, x;
    x = a + b * c;
    printf("Result: x = %d\n", x);

    return 0;
}
```

```
x = a + b * c;
```



High Level Language

```
#include <stdio.h> //header file
for printf()

int main() { //main function
    int a=2, b=3, c=5, x;
    x = a + b * c;
    printf("Result: x = %d\n", x);

    return 0;
}
```

Source Code/HLL



Language
Translator

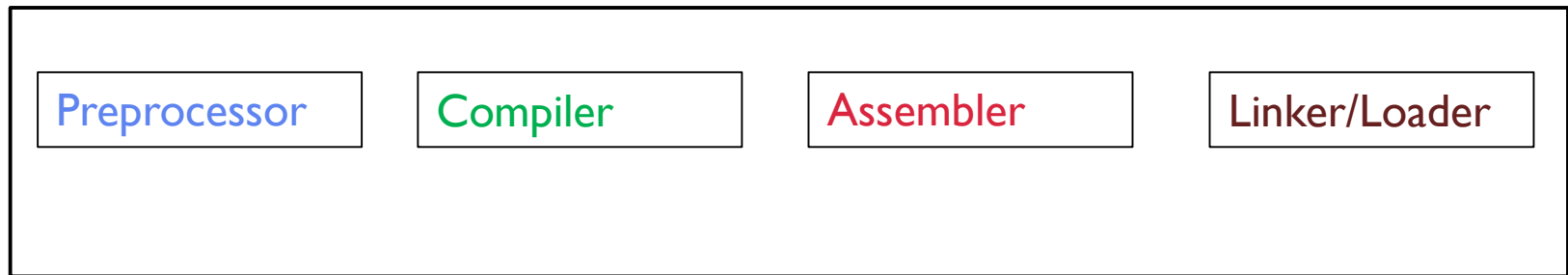


```
0010101010010
10110100101010
11010101010010
100111110010101
01010101010101
11010101010010
100111110010101
```

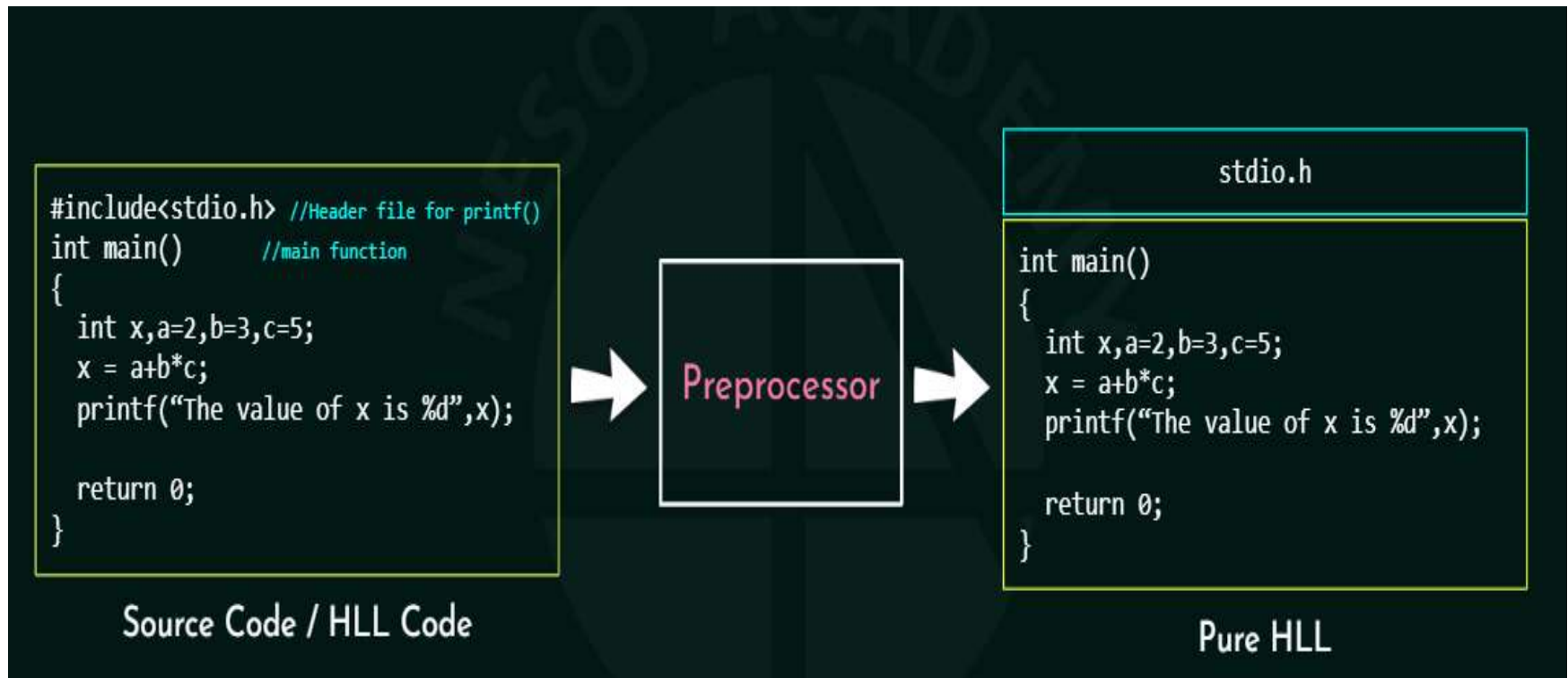
Machine Code



Language Translator-Internal Architecture



Language Translator-Internal Architecture



Language Translator-Internal Architecture

```
stdio.h

int main()
{
    int x,a=2,b=3,c=5;
    x = a+b*c;
    printf("The value of x is %d",x);

    return 0;
}
```

Pure HLL

Compiler

```
.LC0:
.string "The value of x is %d"

main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    mov     DWORD PTR [rbp-4], 2
    mov     DWORD PTR [rbp-8], 3
    mov     DWORD PTR [rbp-12], 5
    mov     eax, DWORD PTR [rbp-8]
    imul    eax, DWORD PTR [rbp-12]
    mov     edx, eax
    mov     eax, DWORD PTR [rbp-4]
    add     eax, edx
    mov     DWORD PTR [rbp-16], eax
    mov     eax, DWORD PTR [rbp-16]
    mov     esi, eax
    mov     edi, OFFSET FLAT:.LC0
    mov     eax, 0
    call    printf
    mov     eax, 0
    leave
    ret
```

Assembly Language

Language Translator-Internal Architecture

```
.LC0:
.string "The value of x is %d"
main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    mov     DWORD PTR [rbp-4], 2
    mov     DWORD PTR [rbp-8], 3
    mov     DWORD PTR [rbp-12], 5
    mov     eax, DWORD PTR [rbp-8]
    imul    eax, DWORD PTR [rbp-12]
    mov     edx, eax
    mov     eax, DWORD PTR [rbp-4]
    add     eax, edx
    mov     DWORD PTR [rbp-16], eax
    mov     eax, DWORD PTR [rbp-16]
    mov     esi, eax
    mov     edi, OFFSET FLAT:.LC0
    mov     eax, 0
    call    printf
    mov     eax, 0
    leave
    ret
```

Assembly Language

Assembler

```
i+0:00101010101001
i+1:0101101001100
i+2:10101101010101
i+3:0100101001101
i+4:11100101010101
i+5:01010101010111
    ⋮
```

Relocatable
Machine Code

Language Translator-Internal Architecture

```
i+0:001010101001
i+1:0101101001100
i+2:10101101010101
i+3:0100101001101
i+4:11100101010101
i+5:0101010101011
⋮
```

Relocatable
Machine Code



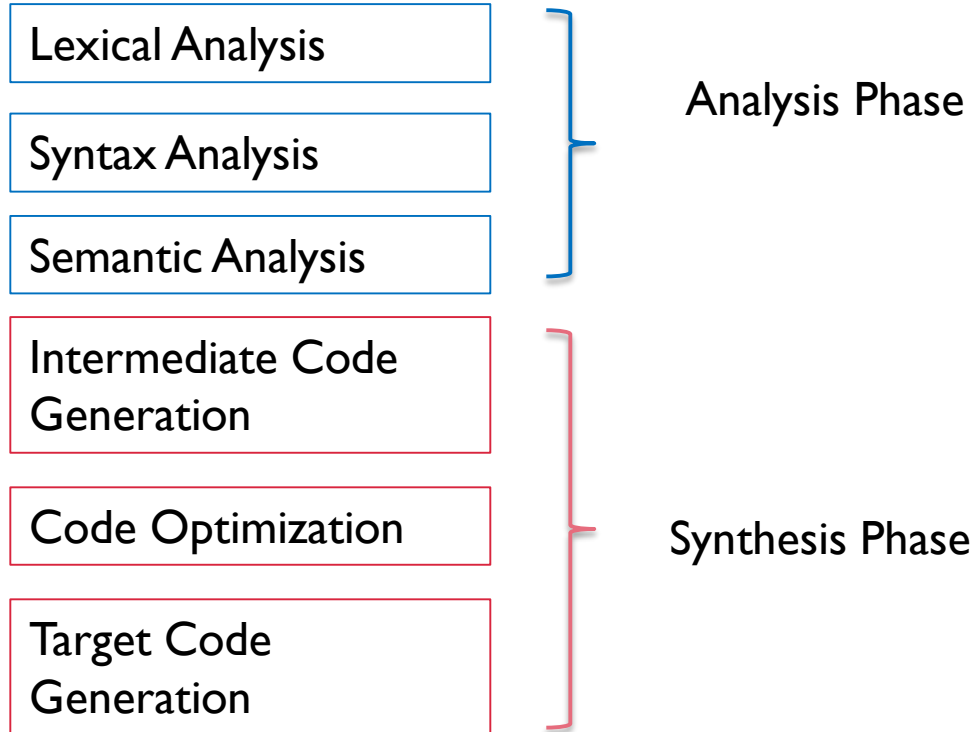
Linker/Loader



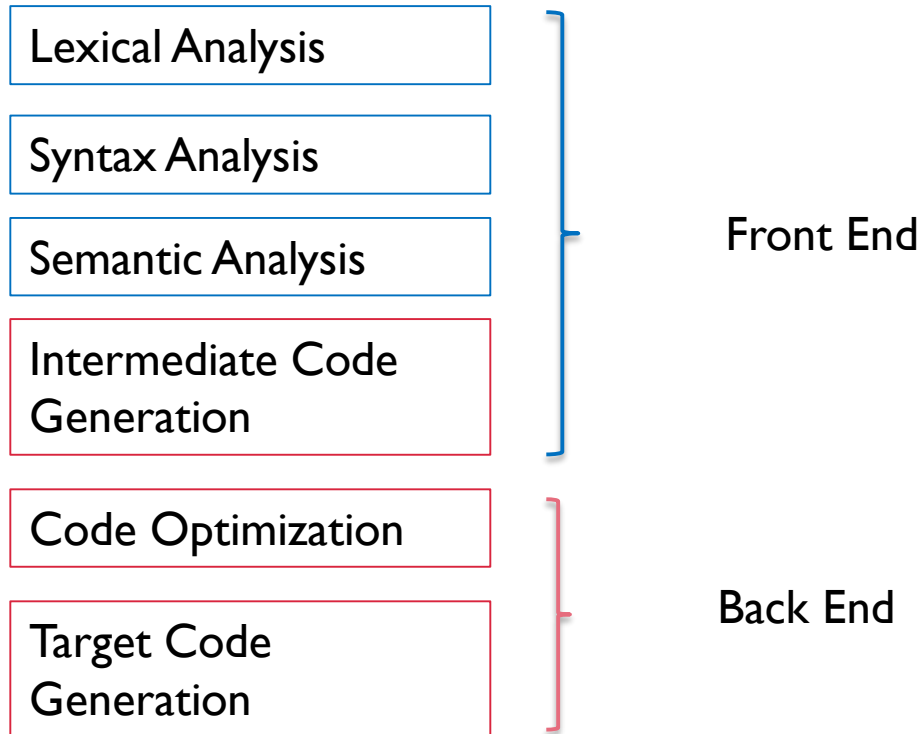
```
0x000004B8: 001010101001
0x000004B9: 0101101001100
0x000004BA: 10101101010101
0x000004BB: 0100101001101
0x000004BC: 11100101010101
0x000004BD: 0101010101011
⋮
```

Absolute
Machine Code

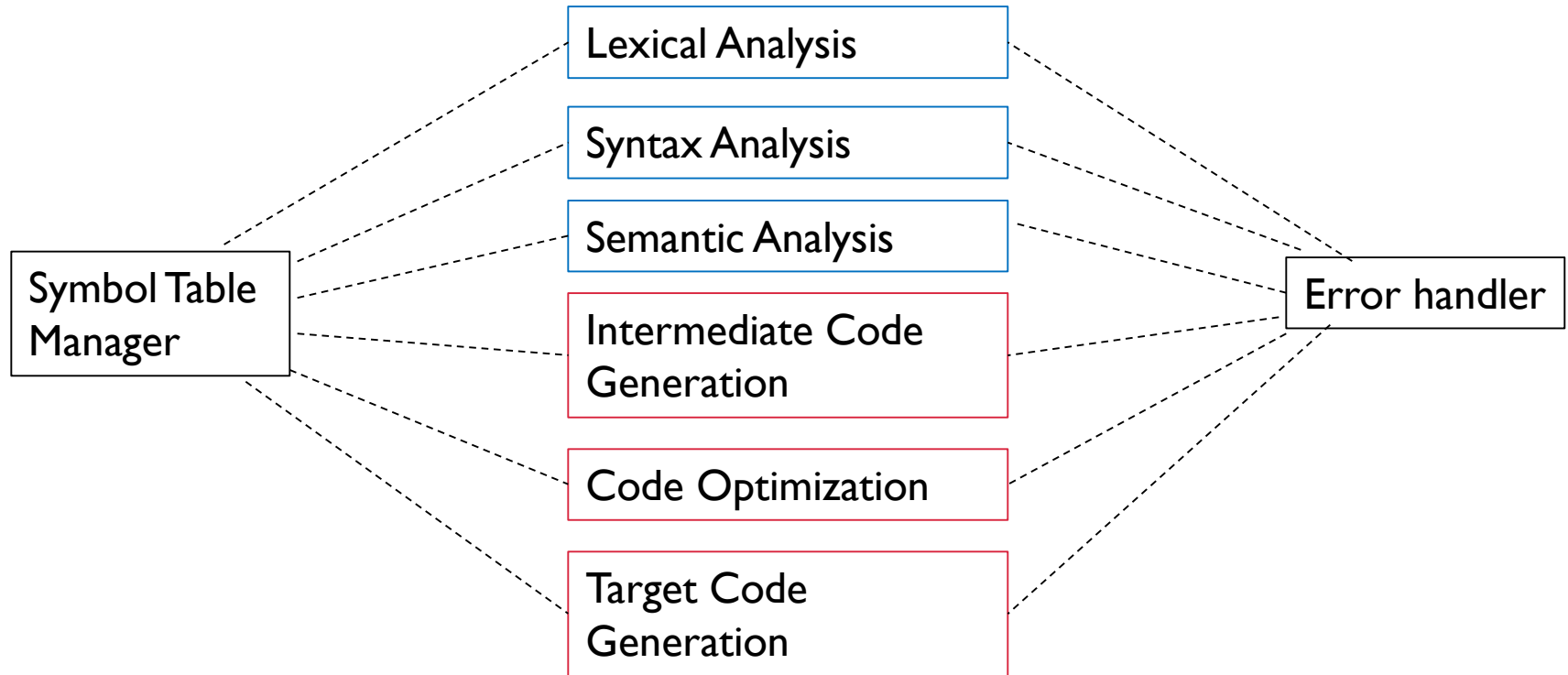
Internal Architecture of Compiler



Internal Architecture of Compiler



Phases of a Compiler



Phases of a Compiler

`x = a+b*c;`

Compiler

```
⋮  
mov     eax, DWORD PTR [rbp-8]  
imul    eax, DWORD PTR [rbp-12]  
mov     edx, eax  
mov     eax, DWORD PTR [rbp-4]  
add     eax, edx  
mov     DWORD PTR [rbp-16], eax  
⋮
```

Lexical Analysis

Syntax Analysis

Semantic Analysis

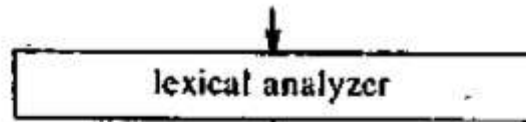
Intermediate Code
Generation

Code Optimization

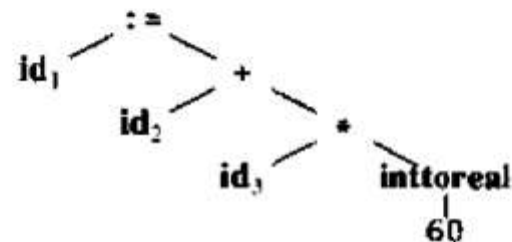
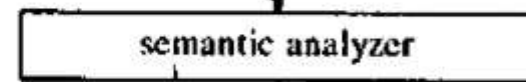
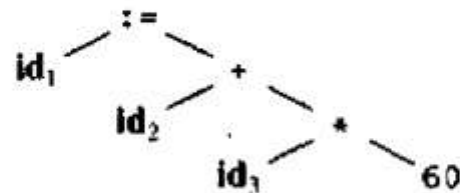
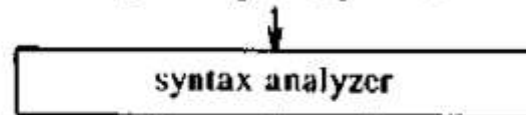
Target Code Generation

Phases of a Compiler

`position := initial + rate * 60`



`id1 := id2 + id3 * 60`



SYMBOL TABLE

1	position	...
2	initial	...
3	rate	...
4		

Phases of a Compiler

intermediate code generator

```
temp1 := inttoreal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```

code optimizer

```
temp1 := id3 * 60.0
id1 := id2 + temp1
```

code generator

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

SYMBOL TABLE

1	position	...
2	initial	...
3	rate	...
4		

References

- ▶ <https://www.nesoacademy.org/cs/12-compiler-design>
- ▶ Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Addison-Wesley, 2006